

Московский авиационный институт
(национальный исследовательский университет)

Факультет информационных технологий и прикладной
математики

Кафедра вычислительной математики и программирования

Лабораторные работы №4 по курсу «Дискретный анализ»

Студент: Т. Д. Пупыкин
Преподаватель: А. А. Кухтичев
Группа: М8О-206БВ-24
Дата:
Оценка:
Подпись:

Москва, 2026

Лабораторная работа №4

Задача: Необходимо реализовать один из стандартных алгоритмов поиска образцов для указанного алфавита.

Вариант алгоритма: Поиск одного образца при помощи алгоритма Кнута-Морриса-Пратта.

Вариант алфавита: Слова не более 16 знаков латинского алфавита (регистронезависимые).

Формат ввода Искомый образец задаётся на первой строке входного файла.

В случае, если в задании требуется найти несколько образцов, они задаются по одному на строку вплоть до пустой строки.

Затем следует текст, состоящий из слов или чисел, в котором нужно найти заданные образцы.

Никаких ограничений на длину строк, равно как и на количество слов или чисел в них, не накладывается.

Формат вывода В выходной файл нужно вывести информацию о всех вхождениях искомых образцов в обрабатываемый текст: по одному вхождению на строку.

Для заданий, в которых требуется найти только один образец, следует вывести два числа через запятую: номер строки и номер слова в строке, с которого начинается найденный образец. В заданиях с большим количеством образцов, на каждое вхождение нужно вывести три числа через запятую: номер строки; номер слова в строке, с которого начинается найденный образец; порядковый номер образца.

Нумерация начинается с единицы. Номер строки в тексте должен отсчитываться от его реального начала (то есть, без учёта строк, занятых образцами).

Порядок следования вхождений образцов несущественен.

1 Описание

Требуется реализовать поиск одного или нескольких образцов в тексте с использованием алгоритма **Кнута-Морриса-Пратта** (КМП) с построением функции переходов через Z -функцию по методу Гусфилда.

Основная идея алгоритма КМП заключается в том, что при каждом несовпадении символа не требуется начинать поиск заново. Вместо этого используется функция неудач F , которая позволяет совершать максимально длинные сдвиги паттерна, используя информацию о его строении.

Для эффективной работы с потоками данных большого объёма используется **потокковая обработка**: текст читается построчно, разбивается на слова, и каждое слово обрабатывается без накопления всех данных в памяти.

Поскольку паттерн состоит из нескольких слов, а вхождение может начинаться с последнего прочитанного слова буфера, применяется **кольцевой буфер** размером n (количество слов в паттерне) для сохранения метаданных о позициях последних обработанных слов.

Для оптимизации вычисления функции неудач используется алгоритм **Гусфилда**, который на основе Z -функции быстро строит таблицу сдвигов за линейное время $O(n)$.

2 Исходный код

Реализация состоит из нескольких ключевых компонентов, каждый отвечающий за отдельный аспект алгоритма.

1 Структура Pos

Вспомогательная структура для хранения метаданных о позиции слова в тексте. Содержит:

- `size_t line` — номер строки;
- `size_t pos` — номер слова в строке.

2 Класс FArray

Класс, отвечающий за предварительную обработку паттерна. Реализует построение массива сдвигов "функцию неудач" F . Данный подход позволяет за линейное время $O(n)$ вычислить все необходимые переходы для автомата КМП.

Вычисление функции неудач разделяется на 3 основных этапа:

1. Построение Z-блоков (Z-функции): На этом этапе для каждой позиции паттерна вычисляется длина наибольшего общего префикса между всем паттерном и его суффиксом, начинающимся с этой позиции.
2. Построение функции sp' : Данная функция определяет длину максимального строгого префикса паттерна, который одновременно является и его суффиксом для текущей подстроки, при условии, что символы, следующие за этим префиксом и суффиксом, не совпадают.
3. Построение функции неудач F : Финальный этап, на котором на основе полученных значений sp' формируется итоговый массив сдвигов. Функция F указывает, к какому состоянию (или какому символу паттерна) необходимо перейти в случае возникновения несоответствия при поиске.

3 Поточковая обработка и поиск

Основная логика сосредоточена в функции `main` и локальной лямбда-функции `process word`.

Программа работает в следующем порядке:

1. **Чтение и парсинг паттерна:** Первая строка нормализуется (нижний регистр) и разбивается на слова.
2. **Инициализация автомата:** Строится массив F для вычисления переходов.
3. **Процесс поиска:** Каждое слово текста сопоставляется с текущим состоянием автомата.
4. **Обновление состояния:** Переменная p отслеживает количество совпавших слов. При несовпадении используется функция F .
5. **Вывод результатов:** При полном совпадении ($p == n + 1$) координаты извлекаются из кольцевого буфера.

4 Кольцевой буфер

Кольцевой буфер хранит координаты последних n обработанных слов. Это необходимо для корректного вывода позиции начала найденного паттерна в условиях потокового чтения.

Индекс в кольцевом буфере циклически увеличивается: $\text{ring_idx} = (\text{ring_idx} + 1) \% n$. Когда найдено совпадение, начало паттерна находится в позиции $(\text{ring_idx} + 1) \% n$.

main.cpp	
<code>FArray(const std::vector<std::string>& pattern)</code>	Конструктор: вычисление Z-функции и построение таблицы F
main.cpp	
<code>void process_word(size_t l, size_t w)</code>	Лямбда-функция обработки одного слова и обновления состояния автомата КМП
main.cpp	
<code>std::vector<Pos> ring_buffer</code>	Кольцевой буфер для хранения координат последних n слов

```

1  #include <iostream>
2  #include <string>
3  #include <vector>
4  #include <cstdint>
5  #include <algorithm>
6
7  struct Pos {
8      size_t line;
9      size_t pos;
10 };
11
12 class FArray {
13     std::vector<size_t> F;
14
15 public:
16     explicit FArray(const std::vector<std::string>& pattern) {
17         size_t n = pattern.size();
18         if (n == 0) return;
19
20         std::vector<size_t> Z(n, 0);
21         size_t L = 0, R = 0;
22         for (size_t i = 1; i < n; ++i) {
23             if (i <= R) {
24                 Z[i] = std::min(R - i + 1, Z[i - L]);
25             }
26             while (i + Z[i] < n && pattern[Z[i]] == pattern[i + Z[i]]) {
27                 ++Z[i];
28             }
29             if (i + Z[i] - 1 > R) {
30                 L = i;
31                 R = i + Z[i] - 1;
32             }
33         }
34         std::vector<size_t> sp_prime(n, 0);
35         for (int64_t j = static_cast<int64_t>(n) - 1; j >= 1; --j) {
36             if (Z[j] > 0) {
37                 size_t i = j + Z[j] - 1;
38                 sp_prime[i] = Z[j];
39             }
40         }
41         F.resize(n + 2, 0);
42         F[1] = 1;
43         for (size_t i = 2; i <= n + 1; ++i) {
44             F[i] = sp_prime[i - 2] + 1;
45         }
46     }
47
48     inline size_t operator[](size_t p) const {
49         return F[p];

```

```

50     }
51 };
52
53 int main() {
54     std::ios_base::sync_with_stdio(false);
55     std::cin.tie(NULL);
56
57     std::string pattern_line;
58     if (!std::getline(std::cin, pattern_line) || pattern_line.empty()) {
59         return 0;
60     }
61     std::vector<std::string> pattern;
62     std::string temp;
63     for (char ch : pattern_line) {
64         if (std::isspace(ch)) {
65             if (!temp.empty()) {
66                 for (char &c : temp) c = std::tolower(c);
67                 pattern.push_back(temp);
68                 temp.clear();
69             }
70             } else {
71                 temp += ch;
72             }
73     }
74     if (!temp.empty()) {
75         for (char &c : temp) c = std::tolower(c);
76         pattern.push_back(temp);
77     }
78     size_t n = pattern.size();
79     if (n == 0) return 0;
80
81     FArray F(pattern);
82
83     std::vector<Pos> ring_buffer(n);
84
85     size_t ring_idx = 0;
86     size_t line_num = 1;
87
88     size_t p = 1;
89
90     std::string current_word;
91     current_word.reserve(20);
92
93     auto process_word = [&](size_t l, size_t w) {
94         if (current_word.empty()) return;
95
96         for (char &c : current_word) c = std::tolower(c);
97
98         ring_buffer[ring_idx] = {l, w};

```

```

99
100     while (true) {
101         if (p <= n && current_word == pattern[p - 1]) {
102             p++;
103             break;
104         }
105         if (p == 1) break;
106         p = F[p];
107     }
108     if (p == n + 1) {
109         size_t match_start_idx = (ring_idx + 1) % n;
110         std::cout << ring_buffer[match_start_idx].line << ", " << ring_buffer[
            match_start_idx].pos << "\n";
111         p = F[p];
112     }
113     ring_idx = (ring_idx + 1) % n;
114     current_word.clear();
115 };
116
117 std::string line;
118 while (std::getline(std::cin, line)) {
119     size_t word_in_line = 1;
120
121     for (size_t i = 0; i < line.length(); ++i) {
122
123         char ch = line[i];
124
125         if (std::isspace(ch)) {
126             if (!current_word.empty()) {
127                 process_word(line_num, word_in_line);
128                 word_in_line++;
129             }
130         } else {
131             current_word += ch;
132         }
133     }
134     if (!current_word.empty()) {
135         process_word(line_num, word_in_line);
136     }
137     line_num++;
138 }
139 return 0;
140 }

```


5 Анализ сложности

- **Временная сложность предварительной обработки:** $O(n)$, где n — количество слов в паттерне.
- **Временная сложность поиска:** $O(m + n)$, где m — общее количество слов в тексте. Каждое слово текста обрабатывается ровно один раз.
- **Пространственная сложность:** $O(n)$ для хранения паттерна, функции F и кольцевого буфера.

Благодаря кольцевому буферу размером n и потоковой обработке программа способна работать с текстами произвольного размера, не накапливая их целиком в памяти.

3 Консоль

```
root@workspaces:/workspaces/Discrete-Analysis-MAI# cmake -B build
root@workspaces:/workspaces/Discrete-Analysis-MAI# cmake --build build
root@workspaces:/workspaces/Discrete-Analysis-MAI# ./build/lab4/lab4
```

```
cat dog cat dog bird
CAT dog CaT Dog Cat DOG bird CAT
dog cat dog bird
1,3
1,8
```

4 Тест производительности

Для анализа эффективности реализованного алгоритма КМП было проведено сравнение с высокооптимизированным стандартным поиском `std::string::find()`. Тестирование проводилось на различных объемах данных: от 0.5 до 2 миллионов слов.

Тест 1: 500,000 слов

```
tim@tim-swift:~/.../$ ./std <./input.txt
std::string::find() Time: 37.4529 ms
tim@tim-swift:~/.../$ ./my <./input.txt
KMP Time: 57.269 ms
```

Тест 2: 1,000,000 слов

```
tim@tim-swift:~/.../$ ./std <./input.txt
std::string::find() Time: 47.0277 ms
tim@tim-swift:~/.../$ ./my <./input.txt
KMP Time: 110.918 ms
```

Тест 3: 2,000,000 слов

```
tim@tim-swift:~/.../$ ./std <./input.txt
std::string::find() Time: 97.3903 ms
tim@tim-swift:~/.../$ ./my <./input.txt
KMP Time: 202.918 ms
```

Сводная таблица результатов:

Количество слов	500 000	1 000 000	2 000 000
<code>std::string::find()</code>	37.45 ms	47.03 ms	97.39 ms
Алгоритм КМП (авторский)	57.27 ms	110.92 ms	202.92 ms

Анализ результатов: Наблюдается четкая линейная масштабируемость обоих алгоритмов. При увеличении объема входных данных в 4 раза (с 0.5 млн до 2 млн слов), время выполнения увеличивается пропорционально, что подтверждает линейную сложность для обоих подходов.

Преимущество стандартной библиотеки: `std::string::find()` стабильно работает быстрее (в 1.5–2.3 раза). Это обусловлено использованием низкоуровневых оптимизаций:

- **SIMD-инструкции:** Стандартная функция сравнивает сразу несколько символов за один такт процессора.

- **Отсутствие накладных расходов:** Моя реализация тратит время на пословную токенизацию, создание объектов `std::string` и работу с кольцевым буфером, в то время как `std::string::find()` выполняет поиск в "сыром" бинарном буфере.

Особенности реализации: Несмотря на отставание в скорости, алгоритм КМП производит поиск именно по логическим единицам (словам) и использует константный объем дополнительной памяти, что критически важно при работе с потоковыми данными большого объема, которые невозможно целиком загрузить в ОЗУ.

5 Выводы

В ходе выполнения лабораторной работы по курсу «Дискретный анализ» была реализована система эффективного поиска паттерна в текстовых данных на основе алгоритма Кнута-Морриса-Пратта. Основные результаты работы:

- Реализован препроцессинг искомого шаблона с использованием **алгоритма Гусфилда** для построения Z -блоков, на основе которых был сформирован оптимизированный массив сдвигов (массив сильных префиксов).
- Спроектирована архитектура **потокового поиска** (stream processing), позволяющая обрабатывать неограниченные объёмы входных данных без их полной загрузки в оперативную память, что обеспечивает пространственную сложность $O(n)$, где n - количество слов в паттерне.
- Внедрён механизм **кольцевого буфера** для хранения метаданных (номера строк и позиций слов), что позволило эффективно восстанавливать координаты начала вхождения паттерна при его полном сопоставлении.

Список литературы

- [1] Гусфилд, Д. Строки, деревья и последовательности в алгоритмах: Информатика и вычислительная биология : [пер. с англ.] / Д. Гусфилд. — СПб. : Невский Диалект ; БХВ-Петербург, 2003. — 654 с. — ISBN 5-7940-0061-1.
- [2] ГОСТ 7.1–2003. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. — М. : Изд-во стандартов, 2004. — 166 с.